

# SDL3 and MIDI2 Integration Strategy for Teatro GUI

## Background and Goals

**Teatro** is the custom GUI/rendering engine within the FountainAI ecosystem, responsible for turning domain data (scripts, storyboards, logs) into visual artifacts (e.g. SVG, Markdown) <sup>1</sup>. It currently emphasizes **retained-mode**, deterministic rendering with modular separation of core logic and UI. The goal now is to evolve Teatro into a cross-platform *interactive* GUI framework by adopting **SDL3** as the underlying windowing and input backend, and deeply integrating the `midi2` module for real-time MIDI 2.0 interaction. This must be achieved without disrupting existing headless output pathways or module boundaries <sup>2</sup>. In summary, we aim to preserve Teatro's high-level architecture (protocol-oriented design, layered modules) while augmenting it with SDL3 and MIDI 2.0 capabilities <sup>3</sup>.

### Objectives:

- **Cross-Platform GUI with SDL3:** Introduce an SDL3-backed rendering and event loop so Teatro can open windows and handle input on macOS, Linux, and beyond (eventually Windows, iOS, etc.) <sup>4</sup>. SDL3 will provide low-level access to graphics, input, and audio in a unified way, allowing Teatro to remain platform-agnostic <sup>5</sup>. Crucially, this will align with Teatro's retained-mode approach: Teatro will control drawing and state, using SDL3 purely as a backend for window management, GPU context, and event pumping <sup>6</sup>. The high-level API of Teatro should remain unchanged for existing usage (headless rendering), ensuring backward compatibility and deterministic outputs for tests <sup>7</sup>.
- **Real-Time MIDI 2.0 Visualization:** Leverage the `Fountain-Coach/midi2` library to visualize MIDI 2.0 **UMP** (Universal MIDI Packet) streams within the GUI in real time. Teatro's new GUI components will be capable of displaying MIDI events on timelines, musical staff or piano-roll style views, and even live text token streams ("token overlays") as they arrive. For example, if FountainAI produces outputs via Server-Sent Events carried over MIDI2, the GUI could render these streaming tokens live on screen <sup>8</sup>. Timeline views will show MIDI events (notes, controls, timestamps) in sync with playback, providing insight into the sequence data flowing through the system.
- **Interactive Playback & Simulation:** Support user-driven playback and simulation of sequences from the GUI. Timeline scrubbing, play/pause controls, and other interactions in the GUI will emit corresponding MIDI 2.0 messages via the `midi2` module to simulate or drive external sound. For instance, dragging a playback cursor could send appropriate UMP messages to jump to that position (e.g. sending a MIDI **Flex** tempo or JR Timestamp reset), and pressing "play" could stream the timeline's Note On events to a selected MIDI output. This allows the GUI not only to reflect MIDI data, but to become a controller that can drive MIDI devices or software synths in real time.
- **MIDI Configuration UI:** Provide user-friendly GUI options to configure MIDI routing and behavior. The GUI will include panels or overlays to select MIDI output devices/ports, map

control knobs or sliders to specific MIDI CC (Control Change) numbers, choose MIDI channels for tracks, and manage device connections – all powered by the `midiv2` library's capabilities. This ensures that users (or developers during testing) can easily route the Teatro GUI's MIDI output to external instruments or virtual endpoints, adjust mappings (e.g. link an on-screen fader to CC#1 vs CC#11), and monitor MIDI I/O in the GUI. These settings will be made accessible within the GUI so that no manual config file editing is needed, improving usability.

By accomplishing these objectives, Teatro will transition from a static renderer to a full-fledged interactive engine: capable of rendering a screenplay or storyboard with live updates, playing it back in sync with MIDI cues, and providing a visual **and** audible preview of sequences. Crucially, this evolution will be done in an incremental, well-tested manner consistent with the project's rigorous engineering standards (design docs, AGENTS.md checklists, snapshot tests, etc.) <sup>9</sup>. The following sections outline the rationale, design, integration plan, and delivery strategy in detail.

## SDL3 Suitability Analysis

**Why SDL3?** SDL (Simple DirectMedia Layer) is a proven, low-level library for cross-platform multimedia. Version 3.0 brings broad OS support (Windows, macOS, Linux, iOS, Android, even experimental tvOS/VisionOS) – essentially anywhere Swift runs <sup>4</sup>. Using SDL3 means one backend can cover all our target platforms, aligning with FountainAI's goal of wide deployment. This eliminates the need for separate per-OS GUI code paths (e.g. AppKit for Mac, X11 for Linux), greatly simplifying maintenance <sup>10</sup>. SDL is **lightweight** and focused: it provides the “engine room” for graphics, input, and audio, but does *not* impose a UI toolkit or framework on us <sup>11</sup>. This is ideal for Teatro, which implements custom rendering logic (script layouts, timeline visuals) without being constrained by pre-made widgets <sup>12</sup>. SDL gives us a blank canvas (window and GPU context) and an event pump, leaving full control to Teatro's rendering engine – exactly what we need for a retained-mode, custom GUI.

**Capabilities:** SDL3 manages window creation, OpenGL/Metal/Vulkan contexts, 2D drawing primitives, input from keyboard/mouse/gamepad, and audio playback out-of-the-box <sup>13</sup>. It's designed for real-time applications like games and multimedia tools, so it's optimized for performance and low-latency input handling. Importantly, SDL3 has a stable API/ABI promise across the 3.x series <sup>14</sup>, so we can adopt it with confidence that minor version updates won't break our code. It's also battle-tested over decades, which reduces platform-specific bugs – we leverage a mature foundation rather than writing our own event or rendering layer from scratch <sup>10</sup>.

**Headless vs Interactive:** SDL3 can operate in headless modes (e.g. rendering to an offscreen surface) which means Teatro can potentially use SDL for offscreen rendering if needed, without opening a window <sup>15</sup>. In interactive mode, SDL opens a native window and captures real user input, enabling us to build an interactive preview on top of Teatro's rendering <sup>16</sup>. This dual capability aligns with Teatro's existing design: headless CLI usage (exporting SVG, etc.) remains fully supported, and now an interactive GUI mode becomes available using the same underlying engine.

**Performance and Control:** By working at the SDL level, we gain fine-grained control over rendering timing and can utilize GPU acceleration. SDL's rendering API allows drawing textures, lines, shapes with hardware acceleration, and we can manage our own render loop (e.g. target 60 FPS for smooth animations) <sup>17</sup> <sup>18</sup>. This is a lower-level of control than high-level GUI frameworks, which is advantageous for Teatro's needs (complex text layout, syncing with MIDI events, etc.). We can optimize critical paths (e.g. caching rendered text to textures for fast redraw) without fighting a framework's abstraction. SDL's low overhead means we only pay for the features we use, and it doesn't bring in a large dependency chain.

Overall, SDL3 is an **excellent fit** as the backbone of Teatro’s GUI: it’s portable, efficient, and unopinionated about how UIs are structured. It lets us preserve Teatro’s independent rendering approach (no mandated UI controls or scene graph) while adding cross-platform interactivity <sup>6</sup>. All these factors make SDL3 a solid choice for the evolution of Teatro’s GUI, as summarized below:

| Approach                 | Pros                                                                                                                                                                                                                                                                                                                                                 | Cons/Challenges                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SDL3<br/>(Chosen)</b> | <ul style="list-style-type: none"> <li>- Cross-platform backend (Windows, macOS, Linux, etc.) <sup>4</sup></li> <li>- Low-level access to GPU, input, audio for high performance <sup>19</sup></li> <li>- Lightweight: no heavy UI toolkit, gives full control <sup>19</sup></li> <li>- Stable API, well-tested in industry <sup>20</sup></li> </ul> | <ul style="list-style-type: none"> <li>- No native UI widgets (must implement custom controls) <sup>12</sup></li> <li>- Need to maintain a Swift wrapper around the C API (added complexity) <sup>21</sup></li> <li>- C interop requires careful memory management (pointers, allocation) <sup>21</sup></li> </ul>                                                                                                                                                                                                  |
| <b>SwiftGtk</b>          | <ul style="list-style-type: none"> <li>- Rich set of premade widgets (buttons, text fields, etc.)</li> <li>- Mature GUI toolkit (GTK) with accessibility support</li> </ul>                                                                                                                                                                          | <ul style="list-style-type: none"> <li>- Extremely heavy: generated Swift bindings ~300k LOC <sup>22</sup>, large binary and slow build times</li> <li>- Requires GTK libraries on target systems (deployment complexity) <sup>23</sup></li> <li>- API is auto-generated and non-Swifty (feels like wrapping C, not idiomatic Swift) <sup>21</sup></li> <li>- Limited platform support (focused on Linux, X11; macOS via XQuartz) <sup>24</sup></li> </ul>                                                          |
| <b>Tokamak</b>           | <ul style="list-style-type: none"> <li>- SwiftUI-like declarative API in pure Swift</li> <li>- Can target Web (WASM/DOM) and some native (via GTK) – potential for web UI reuse <sup>25</sup></li> </ul>                                                                                                                                             | <ul style="list-style-type: none"> <li>- Still immature, only covers a subset of SwiftUI <sup>26</sup></li> <li>- Desktop “native” mode relies on GTK under the hood (inherits GTK’s issues) <sup>26</sup></li> <li>- Tied to SwiftUI paradigm, which might not fit Teatro’s custom rendering needs <sup>27</sup></li> <li>- Introduces a large dependency with uncertain long-term support <sup>28</sup></li> </ul>                                                                                                |
| <b>Native SwiftUI</b>    | <ul style="list-style-type: none"> <li>- Modern, declarative UI with native look-and-feel on Apple OS <sup>29</sup></li> <li>- Already used in FountainKit’s macOS tools, good dev experience</li> </ul>                                                                                                                                             | <ul style="list-style-type: none"> <li>- Apple-only (no Linux/Windows support) <sup>30</sup></li> <li>- Not designed for heavy custom drawing (our SVG/timeline rendering); e.g. cannot natively render SVG content easily <sup>31</sup></li> <li>- SwiftUI still maturing; lacks some flexibility and has performance concerns for complex layouts <sup>32</sup> <sup>33</sup></li> <li>- Would require a different solution (or no GUI) on Linux, fragmenting our approach <sup>34</sup> <sup>35</sup></li> </ul> |

Table: Comparison of SDL3 vs alternative GUI approaches for Teatro.

As shown above, alternatives either add significant baggage (SwiftGtk), are not sufficiently mature (Tokamak), or fail our cross-platform requirement (native SwiftUI). SDL3 hits a sweet spot: it’s lean and **does not restrict our UI design or tie us to a subset of features**, unlike higher-level frameworks <sup>19</sup>. This allows Teatro to implement exactly what it needs with full control. The project’s internal evaluation concluded that SDL3 best serves our custom-rendering use case, whereas frameworks trying to mimic SwiftUI or use heavy widget toolkits were not suitable <sup>36</sup> <sup>37</sup>. (Other options like Qt or Electron were also considered out-of-scope due to their weight and non-Swift tech stack <sup>38</sup>.)

In summary, **SDL3 is the right tool for the job** <sup>39</sup>. It provides a portable, high-performance foundation on which we can build Teatro's interactive features without compromising the existing architecture.

## Swift 6 Native SDL3 Wrapper Design

To integrate SDL3 into a Swift 6 project, we will build a *Swift-native wrapper layer* within Teatro. Rather than exposing SDL's C API directly, we'll create Swift abstractions (classes/structs) that encapsulate SDL functionality in a safe, idiomatic way. This wrapper serves two purposes: (1) present a clean API to Teatro's internals (and potentially high-level modules) without littering the codebase with C pointers and SDL calls, and (2) allow future maintenance or swapping of backends by isolating SDL-specific code.

**Package and Module Setup:** We will add SDL3 as a dependency in Teatro's Swift Package manifest <sup>40</sup>. If an official SwiftPM package for SDL3 exists, we'll use it; otherwise we may include SDL3 as a binary (XCFramework on Mac, compile from source on Linux). The build system will be configured to locate SDL3 headers and libraries on each platform (e.g. using pkg-config or specifying search paths). As part of this step, CI configuration will be updated to ensure SDL3 is present on the runners (for example, installing via Homebrew on macOS and apt on Linux) <sup>41</sup>.

We will create a new **module** (SPM target), tentatively named `TeatroSDLBackend`, within the Teatro repository <sup>42</sup>. This module will contain all SDL-specific code. It is likely to be an *internal* module (not exported in the public API) – initially used only by Teatro's core and not by external users directly <sup>43</sup>. Organizing it separately enforces a clear boundary: other parts of Teatro (e.g. higher-level rendering logic) will interact through our Swift wrapper interfaces, not via SDL calls.

**Wrapper Components:** Key components of the SDL backend module will include:

- **Window** – A Swift class/struct (e.g. `TeatroWindow`) that wraps an SDL window (`SDL_Window*`). It will manage window creation (`SDL_CreateWindow`), destruction (`SDL_DestroyWindow`), and basic properties like title, size, etc. We'll design initializers and methods to be safe (e.g. use optionals or throw errors if creation fails) and manage the underlying pointer's lifetime automatically (perhaps using a deinitializer or explicit close method).
- **Renderer/Surface** – An abstraction for the drawing context. SDL allows creation of a renderer (`SDL_Renderer*`) associated with a window (for 2D drawing), or using explicit OpenGL/Metal calls. Initially, we can use `SDL_CreateRenderer` for simplicity <sup>44</sup>. A `TeatroRenderer` class could provide methods to clear the screen, set draw color, copy textures, present the frame, etc., mirroring needed parts of SDL's 2D API. Down the line, if we need more advanced rendering (custom shaders or intricate text rendering), we might bypass `SDL_Renderer` and draw directly with a GPU API, but that can be encapsulated behind the same interface.
- **Event Handling** – SDL uses a polling model (`SDL_PollEvent`) to retrieve events (keyboard, mouse, window close, etc.). We'll implement an event pump, possibly as part of a `TeatroWindowManager` or similar utility. For instance, a method `pollEvents()` can call `SDL_PollEvent` internally and translate SDL's C structs into Swift enums or structs that represent events (e.g. an enum `TeatroEvent` with cases like `.quit`, `.keyDown(keyCode)`, `.mouseMoved(x, y)`, etc.). This isolates SDL's raw values and provides the rest of Teatro with a Swift-friendly event model.

- **Threading and Run Loop** – To avoid blocking the main thread (which might be running other tasks or the CLI), we may run the SDL event loop and rendering on a background thread or a dedicated dispatch queue. Swift concurrency (actors or async tasks) could also be used to manage this loop. For example, we might spawn a Task that repeatedly polls events and triggers redraws at a target frame rate (e.g. 60Hz). Care will be taken to ensure thread-safety between this loop and Teatro’s data (perhaps using message passing or locking around shared data like the current scene state). SDL itself is mostly not thread-safe for certain calls, so all SDL interaction should happen on one thread (the “GUI thread”) that we control.
- **Swift Memory Safety:** The wrapper will handle converting between Swift types and C pointers safely. For instance, when obtaining pixel data or audio buffers from SDL, we will use `UnsafeMutableRawPointer` carefully and convert to Swift `Data` or other wrappers to avoid leaks. We’ll encapsulate pointer lifetimes: every `SDL_CreateX` will be paired with an appropriate `SDL_DestroyX` call in our types’ deinit or close methods, following RAII principles. By doing so, we prevent common C pitfalls like memory leaks or double frees. The team is experienced with Swift+C interop, so we will apply best practices (like using `withUnsafePointer` closures, etc.) to keep the wrapper robust <sup>21</sup>.
- **Swift API Design:** Although internal, we want the wrapper’s API to feel natural in Swift. For example, rather than exposing SDL’s bitmask flags or C enums directly, we can wrap options in Swift `OptionSet` or enums. We’ll use naming conventions and types that match Swift 6 guidelines (e.g. using `TimeInterval` for durations instead of raw millisecond `Int`s). This makes the code more maintainable and less error-prone than peppering SDL calls around the codebase. In addition, a well-structured wrapper could later be opened up for public use if we ever wanted to expose a low-level customization layer to advanced users – but initially we’ll keep it internal.

By setting up this dedicated SDL3 backend module, we ensure a clear **layering**: Teatro’s core logic (parsing Fountain scripts, generating layouts, handling MIDI sequences) remains decoupled from the specifics of windowing and display. The engine can “ask” the backend to open a window or draw something without knowing about SDL. This adheres to our modular design approach and keeps the door open to replace or supplement the backend in the future (for example, if years later a SwiftUI backend for Linux appears, we could theoretically add it behind the same interfaces without changing Teatro’s higher levels <sup>45</sup>).

## Architecture Integration and API Boundaries

One of the guiding principles of this evolution is **preserving Teatro’s public API and deterministic outputs**. External modules (like FountainKit or any tool that uses Teatro for headless rendering) should not be affected by the introduction of SDL. To that end, we plan the integration as *additive* and opt-in:

- **Non-Breaking Changes:** Teatro’s existing public types and functions (e.g. `TeatroRenderer.renderScript(...)` -> `SVG`) will continue to work exactly as before, producing the same results bit-for-bit <sup>7</sup>. The SDL-based functionality will be introduced as new APIs or commands rather than changing the behavior of current ones. For example, we might add a new method like `TeatroRenderer.launchLivePreview(fountainScript:)` that opens an interactive window, while `renderScript` still just returns an SVG for headless use <sup>46</sup>. This ensures all automated snapshot tests for rendering continue to pass unmodified – a crucial sanity check that we haven’t regressed the core rendering logic.

- **Retained-Mode Rendering Loop:** Teatro's rendering approach is a retained model where the engine computes a full representation of the scene (e.g. the screenplay layout or storyboard state) and then renders it. We will stick to this model in the interactive setting. That is, we won't suddenly switch to an immediate mode or reactive UI paradigm; instead, the engine will update its internal state and then draw to the SDL window each frame. This maintains determinism and makes it easier to correlate the interactive output with the headless output (they are derived from the same underlying state). The **TeatroRenderAPI** (which provides conversion of inputs to outputs) will remain the authoritative source for how things are drawn. SDL just gives us a place to draw them on the screen.
- **Hiding SDL from Public API:** We will keep SDL details behind the scenes. The existence of SDL3 in the system is an implementation detail – from a user's perspective, they might only see that calling a certain function pops up a window or that an interactive mode is available. We will not surface SDL types or require users to install SDL themselves explicitly. For instance, if distributing Teatro as a binary or package, we can bundle the SDL dynamic library or statically link it, so that it “just works.” The wrapper ensures that from the outside, we expose only Swiftly interfaces. This also means if we later swap SDL for another tech, it won't break user code. In summary, **Teatro's public API remains stable and backend-agnostic** <sup>7</sup>.
- **Integration with Existing Modules:** We will integrate SDL in a way that respects the boundaries of existing modules. For example, Teatro's core will not gain a hard dependency on SDL symbols; instead, the core might call an abstraction (protocol or class) provided by the new SDL backend module. If there is a component like `TeatroSampler` (responsible for stepping through timeline frames or syncing with MIDI), we will inject SDL-based functionality via protocols or delegation, so that the sampler doesn't directly call SDL either <sup>47</sup>. This maintains modularity: one could theoretically run Teatro in a mode without SDL (pure headless) by providing a dummy implementation of those protocols. It also makes testing easier (we can substitute a fake window during unit tests if needed).
- **Public API Safety:** Any new public-facing APIs we add for the interactive features will be carefully designed and documented. We'll mark them clearly (and possibly initially as beta or with cautionary notes) if they are experimental. For instance, if we add `TeatroRenderer.launchLivePreview`, we ensure it has clear semantics (maybe it runs until the window is closed, etc.) and document the thread-blocking behavior (does it block the calling thread or not). We will also ensure that misuse doesn't lead to crashes – e.g. calling interactive preview on a Linux server with no display should either noop or throw a descriptive error (like “No display available”), rather than dereferencing null. To help with this, we might include runtime checks for environment (if `$DISPLAY` is not set on Linux, use SDL's dummy driver or refuse to open GUI).
- **Safety on Headless Environments:** By default, we'll program SDL to use a **dummy video driver** when no GUI is available (this is an SDL feature). Our test runner in CI will set `SDL_VIDEODRIVER=dummy` to allow creating a “window” without an actual display <sup>48</sup>. This way, any code paths that initialize SDL can still run in headless mode for testing or in server environments, without failing. The dummy driver can render offscreen or simply discard graphics, which is fine for non-interactive contexts. This approach means including SDL doesn't compromise the ability to run FountainAI's services in a cloud or headless Linux scenario – the GUI just won't actually appear, but the logic can still execute (if needed) or be skipped.

In essence, we treat SDL integration as a **non-invasive augmentation**: if you don't use the new GUI features, Teatro behaves exactly as before. If you do invoke them, it should feel like a natural extension

of Teatro's capabilities, not a bolted-on hack. This philosophy matches the project's development guidelines of non-breaking, additive changes <sup>36</sup>.

## MIDI2 Integration in the GUI

With SDL enabling interactive visuals, we will tightly integrate the `midi2` module to handle all MIDI 2.0 related functionality in the GUI. This integration has several facets:

### Real-Time MIDI Visualizations

One exciting benefit of combining SDL3 with MIDI2 is the ability to **visualize MIDI data streams live**. We will create GUI components to graphically represent MIDI 2.0 UMP packets as they are transmitted or received. There are multiple scenarios for this:

- **Timeline Event Overlay:** For storyboard timelines or musical sequences, we can display markers or notations at the correct timeline positions for each MIDI event. For example, if a storyboard is accompanied by a MIDI track (say, cue tones or background music), the timeline view in the GUI will show note on/off events, control changes, etc., aligned on a time axis. These could be rendered as icons or bars (similar to a piano roll view where notes are bars, or a controller lane for CC messages). As the sequence plays, a playhead moves along this timeline, and the notes light up when triggered.
- **Live UMP Monitor:** We plan to include a "UMP console" or overlay that lists incoming/outgoing MIDI messages in real time (useful for debugging and transparency). Using the MIDI2 library's decoding, we can show each 32-bit or 64-bit UMP packet in a human-readable form (e.g. "NoteOn ch1 note#60 vel100") and possibly color-code them by type. This is akin to a built-in MIDI monitor. It will help developers see what MIDI data is being sent when they interact with the GUI (for instance, scrubbing might generate a flurry of messages that would be listed).
- **Token Stream Visualization:** FountainAI has a concept of streaming text tokens (for example, AI-generated dialogue or real-time logs). In some configurations, these SSE (Server-Sent Events) token streams are being tunneled over MIDI 2.0 (using UMP "Flex Data" messages as a transport) <sup>49</sup>. The GUI will support visualizing these token streams in real time. We will introduce specialized views, such as a `TokenStreamView`, that subscribe to an async sequence of tokens and render them continuously (e.g. appending words to a text box as they arrive, or highlighting the current word in a script) <sup>50</sup>. The MIDI2 library will be used to parse the incoming UMP *Flex Data* messages into token payloads which the GUI can display. This feature brings the AI's real-time output to life: for instance, as FountainAI generates dialogue via SSE events encapsulated in MIDI, the Teatro window could show the dialogue appearing word by word, with timing aligned via JR timestamps. We'll also show status info like stream progress, any lost packet indications, etc., perhaps via a small "stream status" panel (like a colored indicator for connection quality, leveraging data from MIDI2's handling of ACK/NACK if using MIDI-CI reliability) <sup>49</sup>.
- **Visual Enhancements:** We can overlay visual cues directly on the screenplay or storyboard. For example, if a MIDI event corresponds to a certain line in the script (like a sound cue or a character action), we could highlight that line when the event fires. If using the RenderAPI's planned overlay mechanism, we might render an icon or color change in the SVG output itself for that frame. The SDL live preview would then show that highlight in sync with an actual sound or token. Essentially, MIDI events will not only be logged, but can trigger **visual state changes**

in the GUI (flashing a light, progressing a token bar, etc.), making the GUI experience dynamic and tightly coupled to the underlying data stream.

All these visualizations rely on the **MIDI2** library to provide decoding and structured data. Since **MIDI2** covers the full MIDI 2.0 spec, we can handle any message type: Note On/Off, CC, Program Change, Pitch Bend (which in MIDI2 has high resolution), even SysEx8 messages. The library's robust parsing means the GUI logic can remain high-level (subscribe to a stream of parsed events) rather than dealing with raw bytes. For performance, if high-throughput streams are used (e.g. dense MIDI streams for a song), we'll ensure the visualization code is efficient (perhaps throttling UI updates or aggregating events per frame to avoid UI slowdown). SDL is plenty capable of handling frequent updates (it's meant for games), so the main constraint will be ensuring we don't flood the UI with more data than needed.

In summary, the GUI will give **immediate visual feedback** of MIDI 2.0 activity. This not only helps in debugging and development, but it turns Teatro into an educational tool for MIDI 2.0 – one can see the UMP packets as music plays or as an AI generates messages, bridging the gap between raw binary data and human-understandable visualization. This kind of feature is rarely found in standard GUI frameworks and is a direct payoff of integrating our custom MIDI2 module with the custom GUI.

## Interactive Playback and Control

With SDL providing interactive input handling and a window for output, and MIDI2 providing message generation, we can implement rich playback control features:

- **Play/Pause and Navigation:** The GUI will offer controls to play or pause a timeline (e.g. a storyboard sequence or a MIDI track embedded in a project). When the user presses "Play," Teatro will enter a real-time playback mode: advancing the timeline frame by frame (or tick by tick) and sending MIDI events out through the MIDI2 library at the scheduled times. SDL's high-resolution timers will help schedule these outputs precisely, possibly using MIDI2's Jitter Reduction timestamps to align them if communicating to an external device <sup>49</sup>. Pressing "Pause" would halt the advancement; on resume, it continues from the current point. These controls mirror what a user expects in media playback software.
- **Scrubbing and Seeking:** A timeline scrubber (e.g. a slider or simply clicking on the timeline) allows jumping to an arbitrary point in the sequence. When a seek is initiated, Teatro will adjust its internal state to that time (e.g. if jumping backward, it may reset state or if jumping forward, it may fast-forward internally). Correspondingly, MIDI2 output will reflect the jump: for instance, if scrubbing, we might issue an All Notes Off message to avoid hanging notes, and when landing on the new position, issue the relevant Note On events that should be active at that point. We may also leverage a MIDI **Position** or **Flex** message if appropriate (MIDI 2.0 doesn't have a standard "position pointer" like MIDI 1.0's Song Position Pointer, but we could define a custom SysEx if needed or simply handle it at the application level). The scrubber essentially gives the user random access; our integration ensures the *sound* (or external effect) follows the visual position. This closes the loop making the GUI not just a passive viewer but a controller.
- **Emission of MIDI Events from UI Widgets:** Beyond timeline control, other GUI interactions will generate MIDI. For example, if we include an on-screen keyboard or drum pad (for testing sounds), pressing a key on it will use **MIDI2** to emit the corresponding Note On/Off to the selected output device. An on-screen fader control could send continuous CC messages as you drag it. This could be hugely useful for testing the MIDI2 library in a visual way – e.g. adjusting a slider might send high-resolution 16-bit CC values, showcasing the finer granularity of MIDI 2.0

compared to 7-bit MIDI 1.0. Thanks to the `MIDI2` library, constructing these messages is straightforward (it likely has Swift structs for each message type), so the GUI just calls, for instance, `MIDI2.ControlChange(channel: 1, control: 10, value: 16384)` to send a high-resolution CC#10 message.

- **Audio Playback Synchronization:** If the timeline has an associated audio track (e.g. background audio for a scene), we will use SDL's audio subsystem to play it in sync <sup>17</sup>. SDL can open audio devices and handle playback of WAV/OGG data. We'll ensure that when the user plays the timeline, not only do MIDI events go out, but any audio file associated with the sequence (perhaps specified in the storyboard DSL) is loaded and played. Synchronization can be achieved by using the timeline's timecode: for instance, if starting at 0, just play from start; if seeking to a specific timestamp, seek the audio to that timestamp as well. SDL's audio callback will run on a separate thread, but since we control the timing via our main loop, we can coordinate starting/stopping. Combining audio and MIDI ensures the GUI preview is as close to a "real" playback as possible – imagine scrubbing through a scene and hearing the sound effects change accordingly, while also seeing MIDI-driven events highlight.

Under the hood, implementing these interactive features will require enhancing some **Teatro core logic**: likely the aforementioned `TeatroSampler` or timeline manager will be expanded to handle a real-time loop (driven by SDL) and to interface with the MIDI output. We'll ensure that this is done cleanly via protocols or an extension of the RenderAPI – perhaps adding a concept of a "Player" that can load a sequence (script + optional MIDI/Audio) and expose play/pause/seek functions. The GUI then calls those, and internally it uses MIDI2 to send events and SDL to schedule frames.

Finally, because sending out MIDI in real time can be hardware- or environment-dependent (e.g. if no device is connected, or on Linux no ALSA available), we will degrade gracefully. If no MIDI output is configured, the GUI can still run – it will just skip the actual send calls (or send to a default virtual port that isn't connected). This way, someone can still use the timeline visualization without necessarily producing sound.

## GUI-Based MIDI Configuration

To maximize flexibility, the user (or developer using this tool) should be able to configure MIDI settings on the fly through the GUI:

- **Device Selection:** We'll provide a UI element (perhaps a menu or a pop-up dialog) that lists available MIDI output destinations. On macOS, the `TeatroAppleBridge` in the MIDI2 package provides an interface to CoreMIDI, which can enumerate devices/virtual ports <sup>51</sup> <sup>52</sup>. We can query this at runtime to get names of destinations (like "IAC Driver Bus 1" or connected USB devices). The user can then select one as the active output. Once selected, our code will use the MIDI2 library to open that endpoint and route all outgoing UMP messages to it. We can also allow selection of an "Virtual Output" which we create (using `AppleMIDIBridge` to publish a virtual source) <sup>51</sup> – this is useful if the user wants to connect a DAW or synthesizer to us virtually. For input (if we eventually support inbound MIDI control), we could similarly allow choosing an input source or enabling a virtual MIDI input to receive messages.
- **Channel and Routing Options:** A simple UI could let the user specify which MIDI channels to use for certain content. For example, if the timeline has multiple tracks (maybe representing different instruments or personas), the user could assign track 1 → MIDI channel 1, track 2 → channel 2, etc. By default we might use channel 0 for everything, but advanced users might split

things. Also, routing can include turning certain message types on/off (perhaps a filter where the user can disable sending of active sensing or timing messages if they don't want them). Initially, we might not expose a complex routing matrix, but we'll structure the code to allow easy tweaks – possibly reading from a config if not via GUI.

- **CC Mapping for GUI Controls:** If our GUI includes generic controls (like the aforementioned slider or a dial), we will allow the user to configure what MIDI message that control sends. A small “MIDI Learn” feature could be implemented: e.g., the user clicks “Learn” on a slider, then moves a hardware MIDI knob – our MIDI input (if listening) picks up that CC number and automatically assigns the on-screen slider to send the same CC in future. This kind of feature might be beyond MVP, but even without learn, a dropdown of common CCs (1 Modulation, 7 Volume, 10 Pan, etc.) to choose from for a control would be useful. This turns the GUI into a flexible controller, not just a fixed demo.
- **Saving/Loading Config:** To integrate with FountainAI's workflows, the settings could be persisted (perhaps in a small config file or just left for the user to reselect each run). However, since this is more of a developer tool, we might not need a full preferences system yet. Documenting how to set default device via an env variable or config might suffice (e.g. `TEATRO_MIDI_DEVICE=Foo`).

Implementing a config UI from scratch in SDL (which has no native UI widgets) is non-trivial. To keep things manageable, we might start with very simple UI elements: for example, the device list could be cycled by pressing a certain key (like pressing `D` key cycles through detected devices and prints the selection on screen). Or we utilize the window title or console for feedback initially. Another option mentioned in our notes is using **ImGui** (an immediate-mode GUI library often used with SDL) as a quick way to get UI controls for debug purposes <sup>53</sup>. We could optionally integrate Dear ImGui in a debug mode to get dropdowns and buttons without writing a full widget system. In the long run, we might implement a minimal UI toolkit within Teatro (not unlike how the Terminal UI had to implement some controls). Given the scope, we'll likely opt for a pragmatic approach: basic controls in the first iteration (even if just keystroke toggles or an external config), and improve the polish later. The key is that the architecture *supports* these configurations.

With the above, users will have a clear path to set up their environment: “I open Teatro's preview, go to the MIDI config, select my synthesizer from the list, then hit play and I can hear the soundtrack while seeing the score.” Conversely, a developer could use it to test: “I map this slider to CC74 and see if moving it changes the filter cutoff on my synth.” All of it uses the underlying `midi2` library, showcasing its capabilities (like high-res control values and full MIDI CI support if needed in future). Essentially, we are **exposing the power of MIDI 2.0 through an interactive GUI** so it's accessible and observable.

## Impact on Existing Modules

Integrating SDL3 and MIDI2 into Teatro will have ripple effects (mostly positive) on various modules in the FountainAI project. Here we summarize how key modules are affected:

- **Teatro Core Engine:** The core logic of Teatro (parsing, layout, deterministic rendering) remains largely unchanged. We are adding an *adjunct capability* for it to display and play content interactively. Internally, we'll extend the core where needed – for example, integrating a render loop and timeline player that ties together script state, rendering, and MIDI output. Classes like `TeatroRenderer` may get additional methods for initiating interactive mode, but all existing

methods keep their behavior. One small modification will be initializing SDL in the Teatro core startup if interactive features are invoked. We'll ensure that SDL initialization is done lazily (only when needed) and that quitting SDL is handled cleanly when the program exits. The Teatro core may also include additional hooks, such as callback interfaces that the SDL backend calls when, say, a user presses a key (so that core logic can decide what to do, e.g. advance a frame on spacebar press). These hooks will be added carefully to not interfere with headless usage (perhaps guarded by "if SDL active" checks or simply no-ops when not applicable).

- **TeatroRenderAPI:** The `TeatroRenderAPI` is the public API module that external code can use to drive Teatro (as described in the GUI Render API Blueprint) <sup>54</sup> <sup>55</sup>. This module will likely be updated to expose the new interactive functionality in a controlled way. For instance, the RenderAPI might gain a method to launch a preview window given any supported input. It might also expose an object or callback for live status (so that an external orchestrator could know when the window closes or when playback finishes). Additionally, since RenderAPI was designed to be stable for external use, we have to ensure that any new API is thoroughly documented and doesn't break compatibility. We might initially mark the interactive functions as "beta" or keep them internal to Teatro until they're proven, and then graduate them to the RenderAPI. Another impact is that the RenderAPI tests will be extended to cover new features – e.g. if RenderAPI offers a "renderStoryboard" that now can produce an interactive result or at least produce UMP data, we ensure those outputs are correct <sup>56</sup>. The blueprint already anticipated support for UMP in RenderAPI (notice `RenderResult` has an `ump` field) <sup>57</sup>, so this integration fulfills that promise by generating real UMP sequences during rendering and letting them be played.
- **FountainKit & FountainApps:** FountainKit contains higher-level applications like the macOS Launcher (SwiftUI based) and other tooling. These should mostly remain unaffected by the changes to Teatro, since Teatro is used as a library. However, there are opportunities to enhance those apps with the new capabilities. For example, the Mac Launcher could add a button "Preview in Teatro GUI" for a given Fountain script or storyboard, which calls the new RenderAPI function to open the SDL window. Because that app is SwiftUI, it could either treat the Teatro window as an external process (just launching it), or we might eventually create an `NSView/UIView` that bridges the SDL content so it can be embedded in SwiftUI. The latter is non-trivial but possibly doable (e.g. using SDL's Metal view on macOS). Initially, it might simply spawn the separate window. Aside from that, FountainKit's use of Teatro for headless rendering (e.g. exporting SVG) will continue as before – just linking a newer Teatro version. We will verify that FountainKit's unit tests (if any snapshot tests on outputs) still pass, since we intend not to alter existing output. FountainKit's README and documentation might be updated to mention that the GUI preview feature exists (so developers know they can use it for debugging their scripts visually).
- **MIDI2 Library:** The `MIDI2` module itself remains an independent library, but it becomes more deeply integrated into the workflow. We will add `MIDI2` as a dependency to Teatro (if not already) so that the engine can construct and parse UMP messages. This surfaces a few considerations: versioning of MIDI2 (we'll ensure we depend on a stable release of it, e.g. 0.3.0 as per its README <sup>58</sup>, so that our code works against a fixed API). We might contribute back minor improvements to MIDI2 if needed for the GUI – for instance, if we need a function to list MIDI devices on Linux, we might extend it (or more likely, implement that outside of MIDI2 for now, since MIDI2 primarily covers the protocol and Apple CoreMIDI). The `TeatroAppleBridge` submodule of MIDI2 will be key on macOS – we'll use `AppleMIDIBridge` to send UMP to physical endpoints, and `AppleMIDIReceiver` to get input <sup>51</sup>. We need to initialize those properly (which involves dealing with run loop or dispatch queue for CoreMIDI's callbacks – fortunately CoreMIDI is C-based so it can be used in a Swift process easily). On non-Apple

platforms, since `TeatroAppleBridge` won't work, we might in the short term not support hardware MIDI I/O (other than maybe via RTP-MIDI or other network means). This is a known gap; our strategy will document that "MIDI playback is fully supported on macOS; Linux support for MIDI output will require either a port to ALSA or using network MIDI." We might choose to implement a basic Linux MIDI out using ALSA's sequencer in a future iteration, but it could be complex. Alternatively, we encourage Linux users to use an RTP-MIDI bridge (which our system can already do if it treats network as a virtual CoreMIDI endpoint via `rtmidi` driver). These details will be ironed out, but they don't affect the design per se – it's mostly about scope of support. The MIDI2 library's broader presence in the project means more tests run (its own test suite), but also more power: any future MIDI2 spec updates (property exchange, new message types) will automatically be usable in Teatro. We will coordinate version bumps of MIDI2 carefully so as not to break compatibility.

- **Other Modules (SSE, CLI, etc.):** There is a part of Teatro (or related modules) dealing with SSE streams and their transport over MIDI2 (the SSE-over-MIDI2 agent <sup>59</sup>). The GUI integration goes hand-in-hand with that, as the SSE agent deliverables included GUI components like `TokenStreamView.swift` <sup>60</sup>. Our strategy will align with those efforts: as we build the GUI, we'll also implement those streaming views, enabling live AI outputs to be shown. The CLI tools (like `midi2demo` in the MIDI2 package <sup>61</sup>) remain separate, but we may integrate a CLI command in Teatro (like `teatro preview`) that invokes the GUI from command line. This would allow headless users to at least attempt running the preview (with dummy video if no display). It's a minor extension but improves accessibility (not requiring writing a SwiftUI app to use the feature).

In general, the impact on the project is that Teatro becomes a **richer platform**, but we maintain clear separations: the rendering engine, the MIDI engine, and the orchestration layer (FountainKit) each keep their responsibilities. This integration simply connects them in meaningful ways. By maintaining modularity (through well-defined interfaces and keeping new code in its module), we reduce risk of widespread side-effects. The only widespread changes will be configuration (ensuring SDL is available to various build/test environments) and documentation to educate the team about the new capabilities.

## Implementation Plan and Milestones

We will implement this integration in stages, following an iterative plan that allows us to merge incremental progress without breaking anything. Each milestone will be validated by tests and review, in line with the project's AGENTS.md process for safe, stepwise delivery <sup>62</sup> <sup>9</sup>. Below is a phased plan:

### Step 1: Preliminary SDL3 Integration (Backend Setup)

*Objective:* Bring in SDL3 and verify basic window creation in isolation.

1. **Add SDL Dependency:** Update `Package.swift` to include SDL3. For Apple platforms, use an SDL3 XCFramework or package; for Linux, ensure the build finds SDL3 (possibly via a `pkg-config` or compiling from source) <sup>40</sup>. This step ensures our CI and build can link against SDL.
2. **Module Skeleton:** Create the new `TeatroSDLBackend` target <sup>42</sup>. Stub out core classes (`TeatroWindow`, `TeatroRenderer`, etc.) with empty or dummy implementations. The code should compile and link with SDL symbols (e.g. a call to `SDL_Init` to prove linkage).
3. **"Hello Window" Test:** Write a simple test or example that uses the SDL backend to open a window and immediately close it <sup>63</sup>. For instance, call `SDL_Init`, create a 640x480 window, wait 1 second, then destroy it and quit SDL. This will run on macOS and Linux to confirm SDL functions correctly in our environment. In CI, use the dummy video driver to run this test headlessly <sup>48</sup>.
4. **No Public API Yet:** Do not expose any of this to users yet <sup>43</sup>. This step is purely internal. We'll likely

work in a feature branch and merge into main once this basic integration is proven stable (all existing tests still pass, and the new test succeeds on all platforms).

## Step 2: Basic Window & Event Loop Integration

*Objective:* Get an SDL window opening from Teatro code and a minimal event loop running.

1. **Window and Renderer Implementation:** Flesh out the `TeatroWindow` class to actually call `SDL_CreateWindow` with given title and size <sup>42</sup> <sup>64</sup>. Implement a basic `TeatroRenderer` that wraps `SDL_CreateRenderer` to get a 2D rendering context <sup>65</sup>. Possibly, just clear screen to a color each frame for now.

2. **Run Loop:** Establish how the rendering loop will run. For now, use a simple approach: spawn a background thread (or `DispatchQueue.global()` task) that runs a loop: calls `SDL_PollEvent` in a loop and periodically calls `SDL_RenderPresent` <sup>66</sup>. Aim for ~60 FPS delay. The loop will break if a quit event is received (e.g. window closed). This gives us a continuous render cycle. We'll handle events like window close by setting a flag to exit the loop <sup>67</sup>.

3. **Integrate with Teatro Core:** Provide a way to launch this loop via Teatro. For example, add a static method `TeatroRenderer.preview(script: RenderScriptInput)` that in Step 2 simply opens a window and displays a placeholder (since we haven't integrated real content yet) <sup>68</sup> <sup>69</sup>. We might also add a CLI entry point (`teatro preview <file>`) that calls this. At this stage, the content could be just a background color or the text "Teatro Preview" rendered via SDL (we can use `SDL_Surface` + `SDL_ttf` to draw a simple string for placeholder). The key is connecting the high-level call to our SDL backend.

4. **Basic Input Handling:** Map a couple of basic inputs – e.g., if the user presses the window's close button or hits Escape, we set a flag to exit the loop <sup>67</sup>. Perhaps print key presses to console to prove we get input events. This scaffolding sets up for interactive control in later steps.

5. **Testing:** Write unit tests if feasible (for example, a test that creates a window and immediately closes it via posting a quit event, ensuring no crashes) <sup>70</sup>. Use the dummy driver in CI to simulate an environment with "no real display" <sup>48</sup>. Perform manual testing on Mac and Linux: run the preview, see the window appear, close it, verify it exits cleanly and doesn't hang the process <sup>71</sup> <sup>72</sup>.

6. **Milestone:** After Step 2, we have *minimal SDL-backed windowing working with Teatro* <sup>71</sup> <sup>73</sup>. The system can open a window, handle a basic event, and close without issue. This is a good point to merge into main (behind maybe a feature flag) and possibly tag a minor version (though we might keep it unpublished until more features are done). Internal documentation can be updated here to note how to enable the preview (for developers), and an entry in AGENTS.md added about installing SDL3 for development <sup>74</sup>.

## Step 3: Rendering Teatro Content via SDL

*Objective:* Display actual Teatro-rendered content in the SDL window, and introduce simple interactivity.

1. **SVG/Scene Rendering:** Teatro's normal output for a script is an SVG image. We need to render this into our SDL window. Easiest approach: **rasterize the SVG to a bitmap** and draw it. On macOS, we could use CoreGraphics to render SVG data to an image (perhaps leveraging PDF rendering if SVG can be converted) <sup>75</sup> <sup>76</sup>. On Linux, we might use a library like Cairo or librsvg <sup>77</sup>. However, adding heavy deps is undesirable. Alternatively, since we control the rendering engine, we might modify Teatro core to provide us a list of drawing primitives (rectangles, text runs). For this iteration, we can do something simple: render just key elements. For example, draw the title page or first scene's text using `SDL_ttf` (a font rendering add-on). As a prototype, call `TeatroRenderer.renderScript` to get the structured data (maybe we add an API to get lines of text and their positions). Then use `SDL_ttf` to render those strings onto the window at approximate coordinates <sup>78</sup>. This validates that we can show real content. It might not match the SVG exactly in style, but it should reflect the script content. For storyboards/timelines, if we have an UMP sequence, we might skip detailed drawing and just indicate "Timeline loaded: X events". The goal is to **get something non-trivial on screen** from Teatro's engine <sup>79</sup>.

2. **Direct Drawing vs. Images:** We will assess whether to pursue a direct drawing approach (Teatro

provides shapes/text and we draw via SDL) or integrate an SVG rasterizer. As an interim, we'll likely choose whichever is simpler to implement at this stage (perhaps a quick rasterizer on macOS and a limited subset on Linux) <sup>75</sup> <sup>80</sup>. Visual fidelity can improve later. We mainly want to ensure the pipeline from input -> Teatro layout -> SDL output works.

**3. TeatroPlayer Integration:** Replace the placeholder content in the preview window with real rendered output <sup>81</sup>. For example, after loading a Fountain script, the window should display the first page of the screenplay. If the content is scrollable (more text than window), we should at least render the part that fits. This likely means we need some state for scroll offset and logic to render the visible portion. We might hold an offscreen surface of the full rendered content and blit a portion to the window based on scroll offset. Initially, if implementing that is too much, we may require the window to be resizable or the content to scale down. But implementing a basic scroll is achievable: capture mouse wheel events and adjust an `offsetY`, then redraw (by drawing text starting from that offset).

**4. Interactive Controls:** Introduce simple interactions beyond closing the window <sup>82</sup>. Two key ones: (a) If the content is scrollable, map keyboard (arrow keys/PageUp/PageDown) or mouse wheel to scroll <sup>82</sup>. (b) If this is a timeline (with playback), implement pause/resume on a key (spacebar to pause/unpause) <sup>83</sup> <sup>84</sup>. For now, pause could simply stop advancing frames (we might not yet be playing frames in step 3 unless we already implemented timeline playback – if not, we can simulate by auto-scrolling or by just toggling a label “Paused/Playing”). These interactions require connecting SDL events (like key presses) to Teatro's domain logic. We can do that via a simple event handler: e.g. on `Spacebar` event, flip a boolean that our render loop checks to either advance to next frame or not. This sets the stage for real timeline control once we implement frame-by-frame rendering in Step 4.

**5. MIDI Output (initial):** Begin hooking up MIDI output for playback events. If the content has an associated UMP stream (e.g. a storyboard with a MIDI timeline), when playback is running, use `MIDI2` to send those events out in sync. We might not implement the full scheduling yet, but we can do a basic demonstration: e.g., on pressing “Play”, send a sequence of Note On events (say a scale) to prove that the GUI can generate MIDI. Or if a precomputed UMP track is available (perhaps from `renderStoryboard` output), send each message with a slight delay. This can be refined later, but adding a stub for sending MIDI ensures that pipeline (Teatro -> MIDI2 -> out) is started.

**6. Testing & Validation:** At this stage, manual testing is crucial. We will run a known Fountain script through the preview and compare the on-screen result with the SVG output to check layout fidelity <sup>85</sup>. Font rendering differences will be noted (we should try to use the same font as the SVG uses to minimize discrepancies). We test scrolling works (content moves appropriately, no crashes). For timeline, if we have a sample, we test that pressing play does what we expect (maybe logs “sending MIDI...” or actually produces sound if connected to a device). We also verify that none of these interactive actions affect the headless outputs – e.g. ensure `renderScript` still returns identical SVG as before when called normally. If there are differences (say we had to tweak some layout code), ensure they are backward-compatible or behind flags. We might add some snapshot tests if possible: for example, if we can capture the SDL window pixels for a static frame, compare it with an expected image on CI (though image diffs are tricky across platforms) <sup>86</sup>. At minimum, we'll have a smoke test that runs the preview for a few frames and asserts no errors.

**7. Documentation:** Start writing user-facing docs for the new feature <sup>87</sup>. At this point, we can draft a guide “Using Teatro Interactive Preview” explaining how to invoke it (CLI or API), what the current capabilities are (and limitations). This ensures by the time we release, documentation isn't an afterthought. Also, update internal docs: e.g. developer notes on how to debug the SDL backend, or any gotchas encountered (like “on Linux, ensure X server is running or use dummy driver”).

#### Step 4: Cross-Platform Support and Refinement

*Objective:* Broaden support to more platforms, optimize performance, and add polish to interactive features.

**1. Expand to iOS/tvOS (if targeted):** Evaluate running the SDL backend on iOS and tvOS <sup>88</sup>. SDL3 supports these platforms, but integration might require using the SDL provided UIKit app delegate or

some tweaks (on iOS, SDL can take over the main loop). This is low-priority unless there's a plan to demo Teatro on iPad, but we'll at least ensure the code *can* compile for iOS (even if we don't ship it immediately) <sup>88</sup>. If focusing only on desktop for now, we skip this. (If we do pursue: create an example iOS app that uses Teatro to open an SDL view, and handle touch events mapping to our input events, etc. This would reveal any issues like orientation handling or permission issues for CoreMIDI on iOS.)

**2. Windows Support (future-looking):** Swift on Windows is still nascent, but SDL supports Windows fully <sup>89</sup>. We will structure the code to be Windows-friendly (avoiding Swift/objc runtime assumptions). If possible, we might attempt a Windows build of Teatro with SDL to see if it could work, acknowledging that official support may have to wait for Swift 6's Windows maturity. Ensuring no hard-coded Unix/Apple specifics in our SDL layer will make a future port easier <sup>89</sup> <sup>90</sup>.

**3. Performance Tuning:** Now that we have things rendering, we profile the render loop. We aim for **60 FPS** stable during playback. If we find bottlenecks (e.g. text rendering is slow if we redraw all glyphs every frame), we implement caching – for instance, render each unique line of text once to an SDL texture and reuse it each frame <sup>91</sup>. Use SDL's GPU acceleration: ensure we're using texture batching where possible rather than individual pixel drawing. We also manage memory: monitor if we're leaking any surfaces or textures each frame and fix that <sup>92</sup>. Tools like Instruments (on Mac) or Valgrind (on Linux) can help catch leaks or excessive allocations. We also test on large inputs: e.g. a 120-page script or a timeline with thousands of events, to see how our rendering holds up. If scrolling such a large script is sluggish, we consider strategies (maybe rendering only visible portion + small buffer, etc.). This step is about making the interactive experience smooth and not significantly heavier than needed.

**4. Robustness and Edge Cases:** Handle various edge cases in the GUI: resizing the window (we should redraw content at new size, or at least maintain center) <sup>93</sup>, minimizing or losing focus (pause the render loop perhaps to save CPU), multiple windows (for now we might restrict to one preview at a time, but ensure if someone calls preview twice, either it opens a second window or the call is ignored gracefully) <sup>94</sup>. Test unusual scenarios like the user spamming play/pause or rapidly resizing. Also simulate low-resource conditions if possible (maybe run on an older machine or limit CPU to see if we drop frames – then document that heavy content might drop frames). Ensure that closing the window via OS (Alt-F4 / Cmd+W) triggers the same cleanup as our expected path (we already handle SDL\_QUIT event, but double-check no memory is left allocated, no threads left hanging).

**5. MIDI & Audio Refinement:** By now, we should integrate MIDI output fully for timeline playback. That means scheduling events at precise times. Possibly use a timestamp mechanism: e.g., on each frame, send all MIDI events that fall within that frame's time slice. Or use a separate high-res timer for MIDI (since MIDI events might need higher temporal resolution than frame ticks if frame rate is low). We can leverage MIDI2's JR Timestamp messages for accuracy if needed <sup>49</sup>. Also, finalize audio playback if any – ensure audio thread sync. For MIDI input: consider if we want to allow an external MIDI controller to control the GUI. We could at least allow an external "Start/Stop" via MIDI or mapping hardware knobs to on-screen controls. This is not core, but since MIDI2 library and SDL can both handle input, it's doable. Could be a nice bonus to demonstrate two-way integration (e.g. move a physical mod wheel and see the on-screen slider move).

**6. MIDI Configuration UI:** Implement a basic UI for device selection and mappings as planned. Since this is mainly for user convenience, it comes in this later stage after core functionality is solid. For starters, even a text-based approach is fine: e.g., when the user presses "M" key, print the list of devices to the console with numbers, and pressing number keys selects a device. Or cycle through devices with a key. We could improve it by rendering an overlay window using SDL (a simple rect with text) listing devices, and highlight the current one. The user could navigate with arrow keys and press Enter to select. This would demonstrate the capability without needing a full GUI toolkit. We'll also allow volume control or channel selection via keyboard shortcuts as an interim measure (like press "C" to cycle channel 1-16). These interim solutions will be documented, and if time permits, replaced with a nicer ImGui or custom UI panel.

**7. Internal Testing & Beta Release:** At the end of Step 4, we expect the SDL integration to be **beta-quality** – working on Mac and Linux, basic interactivity implemented, and rendering real Teatro content

reasonably well <sup>95</sup>. We will likely align this with an internal milestone where we invite other team members to test the feature (“cross-platform GUI preview” feature complete) <sup>95</sup>. The version can be bumped (e.g. from 1.x to 1.(x+1).0) to indicate a substantial new feature added <sup>96</sup>. We will make an effort to test on all supported configurations and gather feedback. Any critical bugs discovered (crashes, memory leaks, logical errors in playback) will be fixed before proceeding. This phase might involve rapid minor fixes as testers try different use cases. Once it stabilizes, we proceed to finalizing for public release.

## Step 5: Public API Release and Maintenance

**Objective:** Finalize documentation, public interfaces, testing, and officially release the integrated GUI + MIDI2 feature.

1. **Finalize Public API:** Decide which functions/types we want to expose to end users (developers using Teatro). Possibly introduce a nice, simple API like `TeatroRenderer.launchPreview(input: RenderScriptInput)` that packages all the complexity behind one call <sup>46</sup>. We ensure naming is consistent (maybe it should be called `TeatroPlayer` or similar to indicate interactivity). If there are low-level knobs (like selecting a MIDI device programmatically), we might expose them via an options struct or not at all initially (focusing on the default behavior, with manual device selection via UI). We also consider whether to allow external control: e.g., should a developer be able to get callbacks from the preview (like when user closes window)? If so, we design a delegate or completion handler. All public APIs get thorough documentation comments. Any API that we feel is not ready, we keep `internal` or mark with a warning that it may change.

2. **Documentation:** This step involves completing all forms of documentation <sup>97</sup> <sup>98</sup>. We update Teatro's README to highlight the new GUI capabilities and how to use them. We add a chapter in the docs (if the project has a docs folder or wiki) describing the design and usage of the GUI + MIDI integration – essentially a condensed version of this strategy for end-users and future maintainers. Instructions for setting up (e.g. “On Linux, ensure SDL2 is installed and run with `DISPLAY` set” or “On Mac, the preview uses CoreMIDI so allow the app to access MIDI devices”) will be included. We also include examples: perhaps add a new example target in the repository, like `FountainExamples/TeatroLivePreviewDemo` that can be run to show off the feature with some sample script and MIDI ( <sup>99</sup> mentions adding an example). This gives users a template to follow.

We will explicitly document current limitations or differences: for example, “*Note: The interactive preview's rendering of text may have slight layout differences from the SVG output (line breaks may differ by a word or two) due to different font rendering engines. We plan to improve this in future releases.*” <sup>100</sup>. Or “*Rich text (bold/italic) is not yet supported in the live preview.*” Such transparency manages expectations and provides a roadmap for future improvements.

3. **Comprehensive Testing:** By now, we aim to have a test suite covering major aspects. We will include any new unit tests in the CI (with the necessary environment setup as noted) <sup>41</sup>. Some possible tests: open and close window operations (checking no memory leaks via address sanitizer), a test that runs the playback for a short time and ensures the state advances as expected (for determinism, maybe drive the timeline off a fixed tick and verify after X ticks the frame number is Y). If possible, capture a rendered frame buffer and compare pixel values for known simple cases (like if we render a solid red background, ensure pixel sample is red). We will certainly maintain all existing tests: run FountainKit's test suite to confirm nothing broke <sup>101</sup>. Also, do a “full regression” of FountainAI flows – generate some outputs with the new Teatro to ensure they match previous outputs bit-for-bit <sup>101</sup>. Because our integration is additive, we expect no differences in headless outputs, which is the ideal confirmation.

Additionally, we might consider UI automation tests on macOS (XCTest UI can launch an app and verify a window exists) <sup>102</sup>. This might be overkill for a library, but perhaps an integration test that the preview function returns without error and spawns a window (detected via some log or state) would be useful. On Linux, as mentioned, we'll use a virtual display (Xvfb) to run the GUI tests invisibly on CI <sup>103</sup>. We also mark any tests that are too flaky (timing sensitive image comparisons) to be run in a special mode or excluded, to keep CI reliable <sup>104</sup>.

4. **Code Review & Quality Audit:** Given this is a large feature, we will have multiple maintainers review the code before release. Particular attention will be given to memory management (every `SDL_Create*` paired with `SDL_Destroy*` properly) <sup>105</sup> and concurrency (ensure no data races between the render loop thread and main thread – possibly use Swift concurrency tools to validate, like the thread sanitizer). Security-wise, not much exposure since it's local GUI, but we'll ensure no unsanitized input from external sources goes to SDL (fonts or image data might be potential vectors; we trust SDL and system libraries for those). We'll also cross-check that we followed project conventions (naming, formatting, documentation) – the project emphasizes thorough documentation and consistency <sup>106</sup>, so we want this new code to meet those standards. Possibly run linters or formatters if configured.

5. **Release Packaging:** Once everything is approved, we merge the feature into main (if it hasn't been already by earlier milestones) and tag a new release version. Ensure the package manifest is updated (new dependencies like SDL and MIDI2 with correct versions). We might also produce a pre-built binary for convenience if that's done in this project. The release notes will highlight “SDL3-based interactive preview added” and “Integration with MIDI 2.0 for live playback” as key features. We'll thank any internal testers and note any known issues still present so users aren't surprised. This will be a significant milestone for the project, effectively delivering the first interactive GUI capability for FountainAI's tooling.

Following release, we will remain in a maintenance mode for this feature: addressing any bug reports from the community, monitoring SDL3 upstream changes (though those should be ABI-compatible in 3.x) <sup>107</sup>, and likewise keeping the MIDI2 library up to date with spec revisions (the MIDI2 roadmap mentions tracking spec updates <sup>108</sup>, which we'll pull in as needed). By adhering to this plan, we deliver the integration **safely, iteratively, and with comprehensive support**, fulfilling both the technical goals and the quality standards expected in the project.

## Testing and CI Strategy

Quality assurance is paramount for this integration. We outline specific testing and CI measures to ensure reliability:

- **Deterministic Output Preservation:** A core invariant is that all existing snapshot tests (SVG render outputs, Markdown outputs, etc.) remain green. We will run Teatro's full test suite at each stage to confirm no diffs in outputs <sup>101</sup>. Because we aren't altering the logic that generates SVG/MD, we expect no changes, but CI will catch any unintended side effects. We'll add assertions where applicable to double-check that when the interactive mode is not invoked, nothing in the background is different (e.g. ensure no global state in SDL that could perturb things like font metrics).
- **Unit Tests for New Components:** We will write unit tests for the SDL backend where possible. For instance, test that initializing and quitting the SDL subsystem through our wrapper multiple times doesn't leak or crash (calling it twice should maybe safely no-op the second time). We can simulate events by pushing SDL events into the queue (`SDL_PushEvent`) and verifying our translation to `TeatroEvent` works. We can test utility functions (e.g. converting coordinates or merging text surfaces). These tests can run in a headless mode using `SDL_VIDEODRIVER=dummy` so they don't need an actual display <sup>48</sup>.
- **Integration Tests of GUI (Headless Mode):** On CI, we will utilize virtual display or dummy drivers to run some integration scenarios. For Linux, we can use **Xvfb** (X Virtual Framebuffer) to run an X11 server in memory so SDL thinks it has a display <sup>103</sup>. Our test might launch the

preview in a separate thread and then simulate a close event after a short delay, asserting that it closed properly (i.e. the preview function returns). We might also expose a little internal state for testing – e.g., how many frames rendered or a log of events processed – to verify the loop ran. While we can't easily do pixel-level verification in a headless CI (since rendering might rely on GPU), we can use surrogate signals: if we rendered text "Hello" to the window, perhaps we can have our code output some log or set some flag. Another approach mentioned is using OCR on the rendered image <sup>109</sup>, though that may be too complex for automated tests. Simpler: render known shapes and use SDL's function to read pixels back, then compare checksum.

- **Cross-Platform CI Matrix:** Ensure CI runs on macOS and Linux at minimum. If possible (and if Swift on Windows was attempted), add a Windows job to just compile and maybe run basic tests (it's okay if interactive tests are skipped on Windows initially). The macOS CI can actually open a window, but that might not be feasible in a cloud CI environment (there's no GUI). We might stick to dummy driver on Mac CI as well, or use macOS's ability to run GUI tests in a headless mode (not sure if GitHub Actions allows that). We will test locally on macOS machines for actual GUI behavior.
- **Performance Testing:** We plan some performance benchmarks, even if not in CI, to gauge improvements. For example, measure the time to render 1000 lines of text in SDL vs the old SVG path to ensure it's within acceptable range <sup>110</sup>. We might not automate this in CI (which could be unstable due to varying machine speeds), but we'll do it during development and possibly include a benchmark target in the repo for devs to run.
- **Manual QA and Dogfooding:** We will schedule dedicated manual testing sessions (especially after Step 4, the beta phase) <sup>111</sup>. Team members will run the GUI on their machines with various content: extremely long scripts, complex storyboards with rapid MIDI events, SSE token streams in different network conditions (maybe simulate slight packet loss), etc. We'll use these sessions to observe memory usage over time (does it climb indicating a leak?), UI responsiveness (any lag or stutter?), and correctness (are the visuals matching expected output, are MIDI messages sent at correct times?). Logs will be inspected to ensure no hidden errors (we may add debug logging around important events like "Frame rendered in X ms" or "Missed frame due to delay" to help tuning). Any issues found will be documented and addressed. The team's strong testing culture (unit tests, integration tests, snapshot tests) will be extended to these new interactive parts as much as possible <sup>112</sup>, acknowledging that some aspects (like actual GUI output) are harder to test automatically.
- **Continuous Integration Adjustments:** We will integrate new tests into the CI pipeline gradually, monitoring for flakiness. If certain tests prove unstable (for instance, timing differences causing image comparisons to fail or a test that is sensitive to CPU speed), we might mark them as advisory or run them only on nightly builds vs every PR. The goal is to keep CI reliable. We'll also ensure that CI doesn't have unmet dependencies: as mentioned, install SDL on CI runners, and include the MIDI2 library for linking. The MIDI2 tests themselves (like running `midi2demo`) can also be run to ensure we didn't break anything in the MIDI2 package by our usage – though we mostly use it as a dependency, not modify it.
- **Regression and Acceptance Testing:** Before final release, perform a full regression test suite run (all tests across FountainKit, Teatro, etc.) <sup>101</sup> and an acceptance test using the criteria we initially set. The acceptance criteria (from design docs or AGENTS checklist) would include items like "User can open a window on Mac and Linux and see script content", "Scrubbing timeline sends correct number of Note On events to output", "No crash occurs if window closed during playback", etc. We will verify each and mark the feature ready only when all are satisfied <sup>101</sup> <sup>113</sup>.

- **Ongoing Test Maintenance:** We will add new snapshot references if we decide to include any (like an image snapshot of a rendered frame). Those need to be maintained carefully, as differences in rendering engines (especially text on different OSes) might cause them to differ. If too fragile, we may instead use semantic checks (like number of lines rendered equals X, or certain text is present as substring in the output as a proxy check). The test philosophy is to test what's stable and critical. For example, we might skip pixel-perfect assertions of the whole frame (since fonts can differ), but we might check that after rendering, the window's title is set correctly or that the number of draw calls doesn't explode for a given content (maybe count how many SDL texture creations happen to detect leaks).

Finally, any **known issues** that are not fixed by release will be documented (and possibly have corresponding tests expecting the current behavior so we remember to update when fixed). For instance, if kerning is slightly off making layout differ by a few pixels, we document it rather than block release <sup>100</sup>. This testing strategy, combining automated and manual methods, ensures a high confidence level in the new features and that we maintain the project's reputation for reliability and determinism even as we venture into interactive territory.

## Documentation and Future Maintenance

Documentation will accompany this integration at every level. Developer docs (in the repository) will be updated with a detailed explanation of the SDL3 backend design and how to work with it or troubleshoot it. We'll add an **Architecture Manual chapter** about the GUI, including diagrams of how the SDL backend, Teatro core, and MIDI2 interact. There will also be an **AGENTS.md** entry (see below) that outlines the development plan and checklists for this feature, which serves both as a guide during implementation and historical record for later maintainers.

Given the project's emphasis on thorough documentation and onboarding guides <sup>106</sup>, we will ensure new public APIs are fully commented (using DocC comments in code) and examples are provided. The presence of a multi-chapter manual in Teatro's docs suggests we might integrate the content of this strategy into that manual for continuity <sup>114</sup>. Every major package in the project has its README – we will similarly update Teatro's README to reflect the new capabilities, and add or update any READMEs for GUI-related subcomponents (e.g. if we add a `README.md` for the SDL backend module, explaining usage and any platform-specific notes).

For **maintenance**: We plan to monitor SDL's releases (though SDL 3.x should be stable, if SDL 3.1 or 3.2 adds features we need, we might upgrade). The modular nature of our integration means updating SDL or even swapping it out is feasible without touching higher layers, which future-proofs the GUI. We'll also monitor the MIDI 2.0 specifications and update the MIDI2 library accordingly; since our GUI relies on MIDI2 for correctness in parsing, any changes in the spec (unlikely to break at this stage, but additions could happen) will be pulled in. The project's practice of proposals and checklists for new features <sup>115</sup> will continue – future enhancements to the GUI (like adding more widget types or improving fidelity) should go through the same process to maintain design discipline.

By delivering extensive documentation and following the established maintenance practices, we make sure that the GUI + MIDI integration remains sustainable. New contributors will have a clear guide to how it works (via docs and code comments), and existing developers can easily find how to configure or debug it (via the guides and the in-app help we plan to include). The strong testing foundation will catch regressions as the code evolves (for example, if a future change inadvertently slows down rendering, a benchmark test might flag it).

In conclusion, this strategy not only implements the requested features – it does so in a way that **meshes with FountainAI's development culture**, emphasizing modular design, thorough testing, and clear documentation. Teatro will gain a powerful new dimension (interactive GUI with live MIDI 2.0 support) while staying reliable and maintainable.

## AGENTS.md Specification: GUI + MIDI2 Integration

*(The following is a concrete AGENTS.md-style specification to guide and govern the development and maintenance of the SDL3 GUI integration with MIDI2 in Teatro.)*

# Teatro GUI + MIDI2 Integration Agent

( AGENTS .md )

**Owner:** Teatro Core Team / @Fountain-Coach

**Version:** 0.1 (2025-09-26, Europe/Berlin)

**Scope:** Evolve Teatro into a cross-platform interactive GUI engine using SDL3 for rendering/input and integrating MIDI 2.0 (UMP) streams for real-time visualization and control. Deliver an interactive preview window for FountainAI scripts and storyboards that displays live-rendered content, plays timeline MIDI events, and allows user interaction (playback, scrubbing, MIDI routing config) – all without regressing headless output determinism.

```
id: teatro-gui-midi2
name: Teatro GUI & MIDI2 Interactive Preview
description: >
  Introduces an SDL3-powered GUI backend in Teatro and integrates the MIDI2
  library for real-time MIDI 2.0 support.
  Provides a cross-platform windowing system and rendering loop for Teatro's
  outputs, with live visualization of UMP streams (e.g., storyboard timelines,
  SSE token streams) and interactive controls to emit MIDI messages (playback,
  CC tweaks).
  Maintains backward compatibility (no changes to existing SVG/Markdown
  outputs) and adheres to FountainAI's deterministic rendering and testing
  standards.
entrypoint:
  type: process
  command: swift test && swift run TeatroPreviewDemo # run tests and a demo
  preview as a basic usage check
apis:
  - id: teatro-render-api
    path: Docs/RenderAPI.md
    description:
      Public rendering API extended with live preview capabilities (non-breaking
      additions)
  - id: midi2-core
    path: https://github.com/Fountain-Coach/midi2
    description: MIDI 2.0 Swift library used for UMP encoding/decoding and
    device I/O (CoreMIDI on macOS)
```

policies:

- Preserve deterministic, headless outputs:

interactive mode must not alter existing render outputs or logic.

- Encapsulate SDL3 usage behind internal APIs; do not expose SDL types to public interfaces.

- Maintain cross-platform compatibility (macOS/Linux at minimum; no hardcoded Apple-only code in core paths).

- Ensure memory and thread safety (no leaks or data races; pair every SDL allocation with a free; run SDL on one thread).

- Incremental development with feature flags and thorough testing at each milestone (no big bang merge).

- Extensive documentation and example usage for all new features; update user guides and README.

- Automated testing for new components where possible (use dummy drivers, virtual displays for GUI tests in CI).

- No degradation in performance for non-GUI use; optimize GUI rendering to meet 60 FPS for typical content.

## 1) Architectural Objectives

1. **Unified GUI Backend:** Implement a **Swift-native SDL3 backend** for Teatro, enabling window creation, GPU-accelerated drawing, and input handling in a platform-independent way. This should operate as a module that Teatro's core can call into, preserving the engine's platform-neutral design (e.g. no direct AppKit/WinAPI calls). The SDL integration should allow both onscreen rendering and offscreen (headless) rendering using the same code paths where feasible <sup>16</sup>.
2. **Interactive Rendering Loop:** Establish a robust rendering loop that drives Teatro's output to the screen in real time. Use SDL's event polling and timing functions to tick at ~60 FPS or as needed, while synchronizing with Teatro's internal state updates. This includes handling user input events to control the loop (close window, pause/resume playback) and redrawing the scene based on changes (e.g. scrolling or new data). The loop must be carefully managed to avoid blocking other operations and to exit cleanly on request.
3. **MIDI 2.0 Data Pipeline:** Integrate the `MIDI2` library to create a pipeline where **UMP messages flow between Teatro and the GUI**. This entails:
  4. Decoding incoming UMP streams (from files, network, or live sources) into high-level events that GUI components can visualize (notes, control changes, text tokens, etc.).
  5. Encoding and sending UMP messages when the user interacts with the GUI (e.g., scrubbing timeline, pressing play, using a virtual control) to either a virtual MIDI port or selected hardware device.
  6. Utilizing MIDI2's advanced features (JR Timestamps for tight scheduling <sup>49</sup>, MIDI-CI for device negotiation if needed, etc.) to ensure accurate playback and integration.
7. **Real-Time Visualization:** Develop GUI components for **real-time visualization** of MIDI and timeline data:

8. **Timeline View:** A horizontal timeline that displays events (cue points, notes, token positions) against time. It should support a moving playhead and highlighting of current events.
9. **Token/Stream View:** For streaming text or similar data over MIDI, a view that appends tokens or updates text in real-time as messages arrive <sup>50</sup>.
10. **MIDI Monitor Overlay:** An optional console or overlay that logs MIDI events (with timestamps and decoded meaning) for debug/educational purposes. These visualizations should update smoothly with incoming data and be able to handle high throughput (e.g., many short MIDI messages in succession) without lag.
11. **Interactive Control Elements:** Provide **interactive UI controls** to manipulate playback and MIDI:
  12. Play/Pause/Stop controls to start and stop timeline or sequence playback.
  13. A scrubber or slider to seek to different parts of the sequence.
  14. Possibly step/frame advance buttons for frame-by-frame control (useful for examining a specific moment).
  15. On-screen MIDI controllers (simple piano keys, sliders for CC values) to allow manual MIDI input via GUI. Ensure these controls feed into Teatro's timeline engine and the MIDI output path appropriately (e.g., hitting "Play" sets an internal playing flag and triggers event scheduling).
16. **MIDI Routing & Config UI:** Include a **configuration interface** for MIDI settings:
  17. Listing available MIDI output devices/ports and allowing the user to select one (and open the port via `AppleMIDIBridge` on macOS, or equivalent on other OS).
  18. Selecting MIDI channels or channel mapping for multi-track content.
  19. Mapping GUI controls to MIDI messages (e.g., choose which CC a slider sends).
  20. Toggling features like MIDI input echo or enabling virtual MIDI ports. This interface can start simple (text-based or menu-driven) but should be designed such that it can be extended into a user-friendly dialog in future.
21. **Backward Compatibility:** Throughout the integration, **preserve backward compatibility** and modularity:
  22. Teatro's existing headless functions (SVG/Markdown export) must behave exactly as before (bit-perfect outputs, same performance) when GUI features are not invoked <sup>7</sup>.
  23. The SDL/MIDI functionality should be packaged such that if it's not used (e.g., on a server), it doesn't impose runtime overhead or initialization (lazily initialize SDL when needed).
  24. Keep the new code optional at compile-time if possible (for example, if someone builds FountainKit without GUI support, ensure it still builds/works).
25. Adhere to the project's additive change strategy – do not remove or significantly alter public APIs; only add new ones, and mark them clearly as new.
26. **Performance & Resource Targets:** Aim for efficient use of resources:
  27. Rendering performance: **60 FPS** for typical content (e.g., a screenplay page with moderate text or a timeline with tens of events per second). For extreme cases (very large scripts or heavy MIDI streams), it should degrade gracefully (drop frames if needed but not freeze).

28. Memory footprint: The GUI should not leak memory; any large surfaces or textures (like full-screen text bitmaps) should be reused or freed appropriately. Idle memory usage should be reasonable (e.g., a few tens of MB, not hundreds).
29. CPU usage: When idle (no changes, static screen), the render loop should sleep or throttle to avoid maxing out a CPU core. Use event-driven updates where possible (e.g., if nothing changed and no input, don't redraw full frame).
30. Avoiding latency: Sync MIDI events to visuals tightly (e.g., minimal delay between an event's scheduled time and its drawing on screen), aiming for audio-visual sync under ~20ms.
31. **Testing & Quality Gates:** The development must include comprehensive testing and meet quality gates before release:
32. Write new **unit tests** for the SDL backend (window create/destroy, event translation, etc.) and for any pure functions added.
33. Develop **integration tests** for key scenarios (e.g., run a short preview and verify it terminates correctly, simulate a couple of frames and check the state).
34. Use the existing **snapshot test** framework for any deterministic outputs we can capture (for example, if we can snapshot an offscreen rendered image of a small scene, compare it across versions).
35. Manual testing on all target platforms (Mac, Linux GUI, Linux headless, etc.) as part of release criteria <sup>111</sup>.
36. Ensure CI passes on all platforms with the new tests, and incorporate any needed CI setup (SDL on runners, Xvfb for Linux tests, etc.) <sup>41</sup>.
37. No merge to main without passing all tests and reviews (particularly for memory/thread sanitizers).
38. **Documentation & Example:** Deliver extensive **documentation**:
  - Update **README.md** to summarize the GUI feature and how to use it.
  - Add a user guide in `Docs/` (e.g., `Docs/Chapters/TeatroInteractiveGUI.md`) covering setup, usage, and any limitations.
  - Include an **example project or CLI** (`teatro-preview` command or a Swift Playground) demonstrating launching a preview, connecting a MIDI device, etc.
  - Document internal architecture for future maintainers (could be part of AGENTS.md or a design doc) describing how the SDL backend works and how it interfaces with Teatro core.
  - Clearly mark this feature's status (beta/experimental in first release if applicable) and provide guidance on filing issues or contributing improvements.

## 2) Deliverables (PR-ready)

The following deliverables are expected to be completed and merged:

- **Code Modules:**
- `Sources/TeatroSDLBackend/` – **New module** containing the SDL wrapper and related classes:
  - `SDLWindow.swift` (wraps `SDL_Window`; create/destroy, event handling).
  - `SDLRenderer.swift` (wraps `SDL_Renderer` or equivalent GPU context).
  - `SDLEvent.swift` (enum/struct for input events translated from `SDL_Event`).

- `SDLRunLoop.swift` (manages the background thread or main loop for rendering; could also be integrated into `SDLWindow`).
- Platform-specific helpers if needed (e.g., if certain SDL initialization differs on Windows/iOS).
- `Sources/Teatro/Core/TeatroPlayer.swift` – **New internal component** orchestrating interactive playback:
  - Likely a class or set of functions that tie together a `SDLWindow`, `SDLRenderer`, and Teatro's data (script or timeline) plus MIDI output.
  - Handles loading content (calling render functions to get initial data), managing current play state, and updating the view each frame.
  - Connects with MIDI2 to schedule or send events at appropriate times.
- `Sources/TeatroRenderAPI/` – **Updates** to expose new functionality:
  - Possibly add `public func launchPreview(...)` or a `TeatroPreviewController` class in the RenderAPI for external use.
  - Ensure this is added without breaking existing API; maybe under an `#if canImport(SwiftUI)` or always available depending on design.
  - Update any relevant data models, e.g., if `RenderResult` should include more info (though it already has an `ump` field for MIDI data)<sup>57</sup>.
- `Sources/FountainCLI/TeatroPreviewCommand.swift` – **New CLI command** (if CLI exists in the repo):
  - Implements a command like `teatro preview <script.fountain>` to invoke the GUI preview from terminal.
  - Useful for quick testing and for CI (running headlessly to ensure no regressions).
- `Sources/MIDIIntegration/` – **Extensions or utilities bridging to MIDI2**:
  - E.g., `MIDIOutput.swift` that wraps `AppleMIDIBridge` to send UMP, and possibly a stub for Linux.
  - `MIDIEndpointManager.swift` to list devices (on Mac via CoreMIDI, on Linux maybe using an environment variable or placeholder if no support).
  - These can live in Teatro's repo (could be in Teatro core or a new subfolder) to avoid altering the MIDI2 repo.
  - Also include conversion logic if needed (e.g., if timeline events in Teatro are not already UMP, convert them to UMP messages using MIDI2 library types).

• **Visual Components:**

- `Sources/TeatroSDLBackend/Overlay/` – (Optional) If creating any debug UI or overlays, include them:
  - e.g., `MIDIOverlay.swift` that takes decoded MIDI events and renders text or simple graphics to show them. This might use basic SDL text (`SDL_ttf`) or shapes (like rectangles for notes).
  - `TokenStreamView.swift` (if not already in codebase from SSE work) – possibly as a SwiftUI view if integrated with SwiftUI, or in SDL context, maybe rendered via textures. Since our focus is SDL, if SSE token streaming is integrated via SwiftUI previously<sup>60</sup>, we may either reuse that logic or implement equivalent drawing in SDL.
  - Basic UI elements for config: if not using ImGui, perhaps a simple menu bar drawn at top of SDL window with available commands (this could be as basic as drawing text like “[D]evice: X, [Space] Play/Pause, [Esc] Quit” as a legend).

- **Testing:**

- `Tests/TeatroSDLBackendTests.swift` – **Unit tests** targeting the SDL backend:
  - Test window create/destroy (maybe create a window with dummy driver and assert a handle was obtained).
  - Test event translation (feed a synthetic `SDL_KeyboardEvent` and verify our `TeatroEvent` is correct).
  - Test the run loop's timing logic with a stub (maybe inject a mock that counts frames).
  - These tests should be conditional to run with dummy driver and not hang.
- `Tests/TeatroIntegrationTests/InteractivePreviewTests.swift` – **Integration tests:**
  - e.g., launch a preview in a background thread with a short script input, then simulate a close event and join the thread, asserting it closed gracefully.
  - Possibly use a headless mode to render one frame and inspect a known outcome (this might be tricky, but maybe we can have the render loop draw a known test pattern when given a special input for testing).
  - Test MIDI integration in isolation: call our MIDI output functions with a virtual destination that echoes back (the MIDI2 library's `VirtualSourceDemo` could help) <sup>116</sup>, verify that sending a note actually is received by the echo. This could be done in unit tests by launching a MIDI2 virtual port and a listener.

- **Documentation & Examples:**

- `Docs/Chapters/12_InteractiveGUI.md` – **New doc chapter** detailing:
  - Overview of SDL3 integration and architecture (how the pieces fit, maybe sequence diagrams of event flow).
  - How to use the feature (for end users): CLI usage, or code snippet if using API, how to select MIDI devices, etc.
  - Limitations and future improvements section.
- Update `Docs/Chapters/11_TeatroPlayer.md` (if exists) to include streaming overlays info (tie-in with SSE agent docs) <sup>117</sup>.
- Update main `README.md`:
  - Add a section "Interactive GUI Preview" with a brief explanation and perhaps a screenshot (if available) of the GUI in action.
  - Mention any required dependencies (SDL, etc.) and link to the docs for more info.
- `Examples/TeatroPreviewDemo/` – **New example:**
  - A small Swift program (or Playground) that uses `TeatroRenderAPI` to open a preview for a hardcoded Fountain script and UMP sequence. This can serve as a integration test and usage example.
  - Include a MIDI file or generate a simple UMP sequence in code (e.g., one note per second) to demonstrate the playback.
  - Provide instructions in a README in that folder for how to run it.
- `AGENTS.md` – **Update** the engineering guide:
  - Summarize that SDL3 is now a dependency and how to set it up for development (e.g., "install SDL3 via Homebrew on Mac before building").
  - Note any changes to development workflow (like needing a display for certain tests, or how to run them headlessly).
  - Add this spec (or a link to it) to the list of agent design docs for historical record.

- Ensure any new process (like a release checklist item “test interactive GUI on Mac & Linux manually”) is captured.
- **CI/CD:**
  - Update CI configuration (GitHub Actions or others) to:
    - Install SDL3 on macOS and Linux runners (brew install libsdl3 / apt-get libsdl3).
    - Set environment variables (SDL\_VIDEODRIVER=dummy, or `xvfb-run` prefix for running tests that open windows on Linux).
    - Possibly split out GUI tests into a separate job to isolate any flaky behavior.
    - Ensure artifacts (if any, like screenshot images) are stored or compared correctly.
  - Include a nightly or periodic job that runs an extended test with the GUI in a more realistic environment (if possible) to catch issues not seen in dummy mode.
- **Milestone Tracking:** (for project management, not necessarily a deliverable file, but ensure the following milestones are met and checked off in AGENTS or issue tracker)
  - *Milestone 1:* SDL backend scaffolded and “Hello World” window appears on Mac & Linux <sup>73</sup>.
  - *Milestone 2:* Teatro can open a window via a new API/CLI and respond to basic inputs (close, keypress) – minimal integration.
  - *Milestone 3:* Actual script content rendering in window, with basic scroll or interaction; MIDI output demo working (internal alpha).
  - *Milestone 4:* Cross-platform polish, performance >50fps on test content, device selection implemented (feature complete; beta release to team) <sup>95</sup>.
  - *Milestone 5:* Documentation done, tests all passing, review complete – ready for public release tag vX.Y.0.

By delivering all the above, we ensure the **GUI + MIDI2 integration** is done to a high standard, with all supporting materials. This agent spec will be updated as needed during development to reflect any changes in scope or design, and upon completion, it will serve as a reference for maintenance (e.g., if upgrading SDL or extending the GUI, future agents can refer to this document).

### 3) Process Compliance and Maintenance

- **Incremental Development:** The feature is developed across multiple pull requests corresponding to the steps/milestones outlined. Each PR should include the relevant tests and docs for that increment and pass CI independently before merge <sup>62</sup>. Feature flags or stubs will be used as needed to avoid breaking main (e.g., initial merges might hide the new CLI command until fully functional).
- **Code Reviews:** At least two maintainers (including one familiar with core Teatro and one with MIDI expertise) will review each PR. No PR merges if it introduces test regressions or if coverage drops significantly.
- **Determinism Checks:** After integration, run the full deterministic test suite (SVG snapshot tests, etc.) to ensure no deviations. If any are found, treat as high-priority bugs unless intentionally introduced (which we plan to avoid).
- **Issue Tracking:** Any known issues or deferred tasks (e.g., “Linux MIDI output not yet implemented” or “Text rendering differences in preview vs SVG”) will be logged in the issue tracker and linked to this agent. They will either be resolved in follow-up agents or clearly documented as limitations.

- **Continuous Improvement:** Post-release, gather feedback from users/developers using the new GUI. If performance in certain scenarios is a problem, schedule optimization work. If crashes occur, address promptly. This agent will be considered done when the feature is stable in the wild for at least one minor release cycle without major complaints.
- **Alignment with Project Goals:** This integration will remain within Teatro's role as a rendering engine (not expanding into a full end-user application) <sup>118</sup> . We will not add features outside this scope (e.g., no complex GUI editors or network clients – those are out of scope per non-goals). This keeps the project focused and the integration manageable.

By following this spec and the policies above, the SDL3 and MIDI2 integration will be delivered in a controlled, transparent manner. We will achieve a significant enhancement to the FountainAI platform – **making it possible to see and hear the output of the system in real time** – while upholding the rigorous standards of determinism, testability, and documentation that the project demands. The AGENTS.md will serve as our contract to these standards and guide the team through a successful implementation and future maintenance of the GUI framework.

|     |     |     |     |     |     |     |     |     |     |     |                                                   |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|---------------------------------------------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8   | 9   | 10  | 11  | 12                                                | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 |
| 30  | 31  | 32  | 33  | 34  | 35  | 36  | 37  | 38  | 39  | 40  | 41                                                | 42 | 43 | 44 | 45 | 46 | 47 | 48 | 53 | 62 | 63 | 64 | 65 | 66 | 67 | 68 | 69 | 70 |
| 71  | 72  | 73  | 74  | 75  | 76  | 77  | 78  | 79  | 80  | 81  | 82                                                | 83 | 84 | 85 | 86 | 87 | 88 | 89 | 90 | 91 | 92 | 93 | 94 | 95 | 96 | 97 | 98 | 99 |
| 100 | 101 | 102 | 103 | 104 | 105 | 107 | 109 | 110 | 111 | 113 | Strategy for Integrating SDL3 into the Teatro GUI |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |

Framework.pdf

file:///file\_00000000433c61f4be872db02e2eae02

49 50 59 60 117 TEATRO\_AGENT\_SSE\_OVER\_MIDI2.md

[https://github.com/Fountain-Coach/Teatro/blob/2ca7bf9bf39244a6e77a6025f5bf6761a5416857/SSE/TEATRO\\_AGENT\\_SSE\\_OVER\\_MIDI2.md](https://github.com/Fountain-Coach/Teatro/blob/2ca7bf9bf39244a6e77a6025f5bf6761a5416857/SSE/TEATRO_AGENT_SSE_OVER_MIDI2.md)

51 52 58 61 108 116 README.md

<https://github.com/Fountain-Coach/midi2/blob/fec93dae9efbb4f1d56984eb0889983962a1dbab/README.md>

54 55 56 57 118 TEATRO\_GUI\_RenderAPI\_Blueprint\_PR\_Checklist.md

[https://github.com/Fountain-Coach/Teatro/blob/2ca7bf9bf39244a6e77a6025f5bf6761a5416857/TEATRO\\_GUI\\_RenderAPI\\_Blueprint\\_PR\\_Checklist.md](https://github.com/Fountain-Coach/Teatro/blob/2ca7bf9bf39244a6e77a6025f5bf6761a5416857/TEATRO_GUI_RenderAPI_Blueprint_PR_Checklist.md)

106 112 114 115 Evaluation of GUI Capabilities in the FountainAI Project.pdf

file:///file\_0000000051146243b4a86e70d7ab1913